ONE8 FOOTWEAR E-COMMERCE DATABASE

DBMS Skill Task: Multi-Table Joins and Reports

Objective: Combine customer, product, order, and payment information; implement INNER JOIN, LEFT JOIN, and RIGHT JOIN; retrieve complete order details; display purchase history; and generate multi-table reports.

Tables and Relationships

Table	Purpose
Customers	Customer details: name, email and city.
Products	ONE8 footwear products, category, price and colour.
Orders	Customer order date and order status.
Order_Details	Products and quantities included in each order.
Payments	Payment method, amount and status.

Relationships: Customers.Customer_ID → Orders.Customer_ID; Orders.Order_ID → Order_Details.Order_ID; Products.Product_ID → Order_Details.Product_ID; Orders.Order_ID → Payments.Order_ID.

Join Types

INNER JOIN: returns only matching rows from both tables.
LEFT JOIN: keeps every row from the left table; unmatched right-side fields are NULL.
RIGHT JOIN: keeps every row from the right table; unmatched left-side fields are NULL.

SQL Queries and Explanations

1. Combine customer, order, product and payment information

```
SELECT c.Customer_Name,o.Order_ID,p.Product_Name,od.Quantity,
       od.Quantity*od.Unit_Price AS Item_Total,pay.Payment_Method,
       pay.Amount,pay.Payment_Status
FROM Customers c
INNER JOIN Orders o ON c.Customer_ID=o.Customer_ID
INNER JOIN Order_Details od ON o.Order_ID=od.Order_ID
INNER JOIN Products p ON od.Product_ID=p.Product_ID
INNER JOIN Payments pay ON o.Order_ID=pay.Order_ID;
```

Shows matching records across the related tables.

2. LEFT JOIN: include orders without payment

```
SELECT o.Order_ID,c.Customer_Name,pay.Amount,pay.Payment_Status
FROM Orders o
INNER JOIN Customers c ON o.Customer_ID=c.Customer_ID
LEFT JOIN Payments pay ON o.Order_ID=pay.Order_ID;
```

All orders remain visible even if payment data is missing.

3. RIGHT JOIN: include customers without orders

```
SELECT c.Customer_ID,c.Customer_Name,o.Order_ID
FROM Orders o
RIGHT JOIN Customers c ON o.Customer_ID=c.Customer_ID;
```

All customers appear, including those who have not placed an order.

4. Retrieve complete order details

```
SELECT o.Order_ID,o.Order_Date,o.Order_Status,c.Customer_Name,c.Email,
       p.Product_Name,od.Quantity,od.Unit_Price,
       od.Quantity*od.Unit_Price AS Item_Total,pay.Payment_Method,
       pay.Amount,pay.Payment_Status
FROM Orders o
JOIN Customers c ON o.Customer_ID=c.Customer_ID
JOIN Order_Details od ON o.Order_ID=od.Order_ID
JOIN Products p ON od.Product_ID=p.Product_ID
LEFT JOIN Payments pay ON o.Order_ID=pay.Order_ID
ORDER BY o.Order_ID;
```

Combines customer, order, product, line total, and payment details.

5. Display customer purchase history

```
SELECT c.Customer_Name,o.Order_ID,o.Order_Date,p.Product_Name,
```

```
       od.Quantity,od.Unit_Price,od.Quantity*od.Unit_Price AS Item_Total
FROM Customers c
LEFT JOIN Orders o ON c.Customer_ID=o.Customer_ID
LEFT JOIN Order_Details od ON o.Order_ID=od.Order_ID
LEFT JOIN Products p ON od.Product_ID=p.Product_ID
ORDER BY c.Customer_ID,o.Order_Date;
```

Lists each customer's orders and purchased products.

6. Customer-wise purchase report

```
SELECT c.Customer_Name,COUNT(DISTINCT o.Order_ID) AS Total_Orders,
       COALESCE(SUM(od.Quantity*od.Unit_Price),0) AS Total_Purchase_Value
FROM Customers c
LEFT JOIN Orders o ON c.Customer_ID=o.Customer_ID
LEFT JOIN Order_Details od ON o.Order_ID=od.Order_ID
GROUP BY c.Customer_ID,c.Customer_Name
ORDER BY Total_Purchase_Value DESC;
```

Uses COUNT, SUM, GROUP BY and COALESCE to summarise purchases.

7. Product-wise sales report

```
SELECT p.Product_Name,COALESCE(SUM(od.Quantity),0) AS Units_Sold,
       COALESCE(SUM(od.Quantity*od.Unit_Price),0) AS Sales_Value
FROM Products p LEFT JOIN Order_Details od ON p.Product_ID=od.Product_ID
GROUP BY p.Product_ID,p.Product_Name ORDER BY Sales_Value DESC;
```

Shows the quantity sold and sales value per product.

8. Payment summary report

```
SELECT Payment_Method,Payment_Status,COUNT(*) AS Number_Of_Payments,
       SUM(Amount) AS Total_Amount
FROM Payments GROUP BY Payment_Method,Payment_Status;
```

Summarises payment counts and amounts by method and status.

9. Find pending or missing payments

```
SELECT o.Order_ID,c.Customer_Name,
       COALESCE(pay.Payment_Status,'No Payment Record') AS Payment_Status
FROM Orders o JOIN Customers c ON o.Customer_ID=c.Customer_ID
LEFT JOIN Payments pay ON o.Order_ID=pay.Order_ID
WHERE pay.Payment_Status='Pending' OR pay.Payment_ID IS NULL;
```

Identifies orders that may need payment follow-up.

Sample Data

The SQL file creates five sample customers, six ONE8 footwear products, five orders, order line items, and four payment records. A customer without an order and an order without a payment are included to demonstrate outer joins.

How to Run

1. Open MySQL Workbench and connect to MySQL.
2. Open the accompanying SQL file.
3. Review the DROP TABLE commands: they remove and recreate the sample tables, so back up any existing data first.
4. Execute the script and run each SELECT query to view the reports.

Conclusion

This task demonstrates multi-table relationships and INNER, LEFT, and RIGHT JOIN operations. It retrieves complete order information and purchase history, and produces customer, product sales, and payment reports for business analysis.